Dokumentasjon for Prøve Fagprøve (Innlandet Arrangementer)

Skrevet av: Kristian Haugsrud

Dokumentasjon for Prøve Fagprøve (Innlandet Arrangementer) 1
Oppgaven 4
1.Den lokale arrangementsoversikten 4
1.1 Funksjoner 4
1.1.1 Arrangementer som innhold 4
1.1.2 Registrering og innlogging 4
1.1.3 Lagre favoritter 4
1.1.4 Utforske arrangementer 4
1.1.5 Beskyttede ruter 4
1.2 Generelle krav 4
2. Arbeidsform og prosess 5
Planlegging 5
Min tolkning av oppgaven 5
2. Krav til applikasjonen 6
2.1. Brukerhistorier
For å sikre at løsningen møter brukernes behov, har jeg definert følgende kjerne-historier: 6
3. Generelt 6
4. Generelle tanker om oppgaven 6
Techstack 7
Frontend 7
Backend og Database 8
Vurdering av arkitektur og serverlogikk 8
Drøfting av autentiseringsmetode 8
Verktøy/applikasjoner 9
Autentisering 9
Oversikt over roller og tilgang 9
Implementasjon av sikkerhet 10
Arkitektur 10
1. Frontend 10
2. Backend 11
Database struktur 11
Tabelloversikt 11
ER-Diagram (Entity Relationship)| 11
Relasjoner 12
Dag til dag plan 13
Torsdag 19. mars 13
Fredag 20. mars: 13
Mandag 23. mars 13

Tirsdag 24. mars 14
Onsdag 25. mars 14
Torsdag 26. mars 14
Fredag 27. mars 14
3. Gjennomføring 14
3.1 Torsdag 19. mars (dag 1) 14
Egenvurdering og refleksjon 15
3.2 Fredag 20. mars (dag 2) 15
Egenvurdering og refleksjon 16
3.3 Mandag 23. mars (dag 3) 16
Egenvurdering og refleksjon 18
3.4 Tirsdag 24. mars (dag 4) 19
Egenvurdering og refleksjon 20
3.5 Onsdag 25. mars (dag 5) 20
Egenvurdering og refleksjon 22
4. Egenvurdering 23
Hva jeg har lært 23
Hva jeg kunne gjort annerledes 24
Planleggingen 24
Gjennomføringen 24
Plan for videre utvikling (Hvis jeg hadde hatt mer tid) 24
Brukerinnstillinger 24
Utvidet Admin-grensesnitt 25
Påmeldingssystem og sosial bevisføring 25
Interaktivitet og kommentarer 25
Varslinger og påminnelser 25
Kartvisning 25
Læremål 25
Planlegge, utvikle og dokumentere løsninger med innebygd personvern og sikkerhet 25
Utvikle og bruke dokumentasjon og veiledninger 25
Håndtere påloggingsopplysninger på en sikker og forsvarlig måte 26
Velge og bruke relevante rammeverk og moduler til utvikling 26
Feilsøke kode og rette feil i algoritmer og kode 26
Utvikle og tilpasse brukergrensesnitt som ivaretar krav til universell utforming 26

Oppgaven

1.Den lokale arrangementsoversikten

Det skjer mye i Innlandet, men det er spredt over mange kanaler og vanskelig å få oversikt over. Din oppgave er å bygge en webapplikasjon hvor brukere kan utforske lokale arrangementer og lagre sine favoritter. Tenk over hvem som bruker en slik tjeneste — en som planlegger helgen, en arrangør som vil vises, eller en kommuneansatt som vil promotere lokalmiljøet?

1.1 Funksjoner

1.1.1 Arrangementer som innhold

Arrangementer administreres i et redaktørstyrt dashboard. Tenk grundig over hvilke felter som er nyttige (tittel, dato, sted, kategori, bilde osv.) og argumenter for valgene dine i dokumentasjonen.

1.1.2 Registrering og innlogging

Applikasjonen skal ha autentisering via en authentiserings-plattform. Brukere skal kunne opprette konto og logge inn. Drøft valget av autentiseringsmetode (e-post/passord, magic link, OAuth) — hva passer best for målgruppen?

1.1.3 Lagre favoritter

Innloggede brukere skal kunne lagre arrangementer som favoritter og se en liste over disse. Her kobler du brukeren til data i databasen — tenk over tabellstruktur og sikkerhet (Row Level Security).

1.1.4 Utforske arrangementer

Alle besøkende kan se og filtrere arrangementer uten å være innlogget. Begrunn arkitekturvalgene dine (hva hentes fra hvor, og hvorfor?). Dette er kjernen i applikasjonen — brukeropplevelse og ytelse teller her.

1.1.5 Beskyttede ruter

Sider som krever innlogging skal være beskyttet. Hvordan du gjør dette er valgfritt men du må begrunne valget.

1.2 Generelle krav

Responsivt design — tenk mobil
TypeScript gjennom hele kodebasen
God og vedlikeholdbar kode med dokumentasjon

Kildekode tilgjengelig i Git gjennom hele perioden
Applikasjonen skal være deployet og tilgjengelig fra internett
Beskriv og begrunn arbeidsmetodikken din

2. Arbeidsform og prosess

Planlegg med brukerhistorier og tasks — Jira eller tilsvarende. Oppgaven vurderes på planlegging, gjennomføring, dokumentasjon og egenvurdering. Du forventes å kunne argumentere for teknologivalgene dine på tvers av stacken (for eksempel, hvorfor Sanity til innhold og Supabase til brukerdata). Selvrefleksjon og helhetlig forståelse teller like mye som det som faktisk kjører i nettleseren.
Lykke til!

Planlegging

Min tolkning av oppgaven

Jeg skal lage en webapplikasjon hostet på nett som viser en oversikt over alle arrangementer som skjer i Innlandet. Brukere skal kunne søke gjennom alle arrangementer og lagre favoritter som vises i en egen visning. Arrangører skal kunne opprette og redigere oppkommende arrangementer.

Jeg skal bygge webapplikasjonen med arrangørere og brukere i tankene. Prøve å sette meg i deres sko for å se hva som er viktig for en bruker som planlegger helgen sin, hvor lett er det å finne/filtrere gjennom arrangementer, lagre favoritter og hvilken informasjon som er viktig for arrangementene.

Webbapplikasjonen skal ha mulighet for innlogging. Det er åpent hva brukeren skal bruke for å opprette bruker å logge inn, men det må passe for målgruppen. Du skal logge deg inn som bruker for å kunne lagre favoritter og se en liste over alle arrangementer de har satt som favoritt. Du skal kunne logge deg inn som arrangør for å opprette eller endre oppkommende arrangementer. Det er åpent om arrangører skal dele et dashboard for å opprette arrangementer eller om det skal være unikt per arrangør.

Brukere skal lett kunne søke og filtrere gjennom arrangementer. Brukere skal kunne lagre favoritter og se dette i en liste. Sider som krever innlogging skal være beskyttet, det er valgfritt hvordan siden skal være beskyttet. Rammeverk og teknologi er valgfritt.

Applikasjonen skal demonstrere grunnleggende funksjonalitet som kreves i moderne webapplikasjoner. Det jeg legger i dette er at jeg skal vise at jeg mestrer kravene i oppgaven som responsivt design, god brukeropplevelse, ytelse og sikkerhet. Jeg vil også ha fokus på universell utforming og personvern (GDPR) da dette er viktig for en profesjonell løsning. For å

sikre fremdrift vil jeg jobbe agilt og fokusere på en god MVP, slik at kjernefunksjonaliteten fungerer optimalt.

2. Krav til applikasjonen

Applikasjonen skal være responsiv og gi en god brukeropplevelse på både mobil og desktop. Koden skal være av god kvalitet, typesikker (TypeScript) og lett å vedlikeholde. Jeg står fritt til å velge verktøy.

Prosjektet skal planlegges nøye med brukerhistorier og tasks i Jira eller tilsvarende verktøy. Kildekoden skal være tilgjengelig i Git/Github gjennom hele perioden, og den ferdige løsningen skal være deployet og tilgjengelig på nett. Dokumentasjonen skal svare på hvorfor valg av metodikk og teknisk arkitektur er gjort.

2.1. Brukerhistorier

For å sikre at løsningen møter brukernes behov, har jeg definert følgende kjerne-historier:

- **Som en gjest** ønsker jeg å se en liste over arrangementer, slik at jeg kan finne på noe å gjøre uten å måtte logge inn først.
- **Som en bruker** ønsker jeg å kunne trykke på et hjerte for å lagre favoritter, slik at jeg har min egen oversikt over ting jeg vil dra på.
- **Som en arrangør** ønsker jeg å kunne legge ut nye arrangementer med bilde, slik at jeg kan nå ut til flest mulig folk.
- **Som en admin** ønsker jeg å kunne styre hvem som har hvilke roller, slik at jeg har kontroll på hvem som kan legge ut innhold i appen.

3. Generelt

Oppgaven består hovedsakelig av 4 deler, planlegging, gjennomføring, dokumentasjon og egenvurdering.

4. Generelle tanker om oppgaven

Jeg tenker i utgangspunktet at oppgaven er veldig grei og skal kunne løses godt, men jeg ser at det er noen fallgruver jeg må være obs på. Spesielt gjelder det med autentisering og tilgangskontroll.

Siden jeg skal ha både vanlige brukere og arrangører, planlegger jeg å løse dette ved å bruke roller. Arrangører må kunne opprette og redigere innhold, mens vanlige brukere kun skal kunne lagre favoritter. Jeg tenker også å lage en admin-rolle som har full tilgang til alle arrangementer og kan styre roller.

Dette er et punkt jeg kan risikere å sette meg fast i, så jeg må tenke godt gjennom logikken (for eksempel ved bruk av RLS slik at jeg ikke overkompliserer det og mister tid til selve "MVP-en".

Alt i alt er jeg veldig spent på å se hvordan jeg løser disse utfordringene i praksis.

Techstack

Sette opp techstacken som skal brukes til å løse oppgaven. Både for Frontend og Backend

Frontend

I frontend skal jeg bruke Next.js med TypeScript. Jeg har mye erfaring med dette og det er det rammeverket jeg kjenner best. Next.js er et stort og velkjent rammeverk som brukes i alt fra små prosjekter til store bedriftsløsninger. Det er veldig godt dokumentert, noe som gjør det lett å finne gode eksempler og løsninger på utfordringer som dukker opp.

Jeg velger Next.js fordi det gir applikasjonen veldig god ytelse. Ved å bruke "Server Components" kan jeg hente data fra databasen på serveren før siden sendes til brukeren. Dette gjør at siden laster raskt og fungerer godt for de som søker etter arrangementer. TypeScript bruker jeg gjennom hele prosjektet for å ha bedre kontroll på koden og unngå unødvendige feil, noe som er et krav i oppgaven.

Sammen med Next.js bruker jeg en del pakker jeg er godt kjent med og har god erfaring med å bruke:

- Tailwind CSS**:** Dette bruker jeg til styling. Det lar meg bygge et responsivt design raskt og effektivt. Siden oppgaven krever at løsningen skal fungere godt på mobil, er Tailwind et godt valg da det gjør det enkelt å styre hvordan elementer endrer seg på ulike skjermstørrelser.
- Shadcn/UI: Dette er et komponentbibliotek jeg kommer til å bruke for å bygge applikasjonen. Ferdiglagde komponenter for knapper, kort, inputs osv. kommer til å øke hastigheten jeg kan lage UI-en på, slik at jeg kan bruke mer tid til å fokusere på selve brukerflyten og logikken.
- Lucide React: Jeg bruker Lucide for ikoner. Gode ikoner gjør applikasjonen mer intuitiv og lettere å navigere i for brukeren.
- BiomeJs: Jeg bruker Biome til linting og formatering av koden. Det hjelper meg med å holde kodebasen ryddig, fjerne ubrukt kode og sikre at jeg skriver god og vedlikeholdbar kode gjennom hele prosjektet. Det er også mye raskere enn eldre verktøy som ESLint og Prettier, noe som gir en bedre flyt i utviklingen.
- Zod & React Hook Form: Brukes i kombinasjon for å håndtere skjemavalidering. Dette sikrer at dataene arrangører legger inn er korrekte, og gir brukeren gode feilmeldinger i sanntid.
- TanStack Query: Vurderer å bruke det for å håndtere asynkron tilstand på klienten, spesielt for å gi umiddelbar respons når en bruker lagrer et arrangement som favoritt, men har ikke brukt det så mye før så vi får se hva som blir best underveis i utviklingen.

- Date-fns: Siden håndtering av tid og dato er sentralt i en arrangementsoversikt, brukes date-fns for å formatere og vise tidspunkter på en brukervennlig måte.

Backend og Database

Som backend og database har jeg valgt å bruke **Supabase**. Dette er en "Backend-as-a-Service" plattform (BaaS) som bygger på en PostgreSQL-database. Jeg har valgt denne teknologien fordi den lar meg sette opp en profesjonell og sikker backend veldig raskt, slik at jeg kan bruke mer tid på å utvikle funksjonaliteten til applikasjonen.

Jeg bruker Supabase til følgende kjerneoppgaver:

- **Database (PostgreSQL):** Dette er en relasjonsdatabase som er veldig god på å håndtere koblinger mellom data, for eksempel å koble et arrangement til en arrangør, eller en bruker til sine favoritter. Dette sikrer god dataintegritet.
- **Autentisering (Supabase Auth):** Jeg bruker denne tjenesten for å håndtere innlogging og registrering. Den støtter både e-post/passord og sosiale tilbydere som Google og Facebook. Dette er en sikker løsning som følger bransjestandarder, noe som er viktig for å ivareta brukernes personvern.
- **Fillagring (Supabase Storage):** For å håndtere bilder til arrangementene bruker jeg Supabase Storage. Dette gjør at arrangører kan laste opp bilder direkte, som så lagres i skyen og serveres raskt til brukerne via en URL.
- **Row Level Security (RLS):** Dette er en innebygd sikkerhetsfunksjon i databasen som jeg vil bruke for å kontrollere hvem som har lov til å se, endre eller slette data. Dette er en veldig sikker måte å håndtere tilgangskontroll på, da reglene ligger helt nede i databasen og ikke bare i koden.

Vurdering av arkitektur og serverlogikk

Jeg har også vurdert å bruke *Supabase Edge Functions* for serverlogikk, men har valgt å la være for denne oppgaven. Siden jeg bruker Next.js, kan jeg håndtere server-side logikk og database-kall direkte gjennom *Server Actions* og *API Routes*. Dette holder kodebasen mer samlet og gjør arkitekturen enklere å vedlikeholde, samtidig som det er mer enn nok for å dekke kravene til en god MVP.

Drøfting av autentiseringsmetode

Oppgaven ber om en vurdering av autentisering (e-post/passord, magic link eller OAuth). For denne målgruppen har jeg valgt å prioritere e-post og passord som hovedmetode, da dette er det de fleste er mest komfortable med. Jeg har likevel valgt Supabase Auth fordi det enkelt lar meg utvide med OAuth (Google/Facebook) senere. For en lokal tjeneste i Innlandet er det viktig med lav terskel for å opprette bruker, og sosiale påloggingger vil fjerne mye av friksjonen ved registrering.

Verktøy/applikasjoner

For å ha god kontroll på prosjektet og sikre en strukturert arbeidsprosess, vil jeg bruke følgende verktøy:

- Jira: For prosjektstyring, oppretting av brukerhistorier og sporing av oppgaver.
- Git & GitHub: For versjonskontroll og lagring av kildekode.
- Cursor: Min foretrukne kodeeditor for selve programmeringen.
- Slack, Teams & Outlook: For kommunikasjon med kollegaer, veiledere og sensorer.
- Dbdiagram: For visualisering av database tabeller og koblinger
- Supabase Dashboard: For administrasjon av database, autentisering og lagring.
- Gemini: Brukes som støtteverktøy for ideutvikling, problemløsning og kvalitetssikring av tekst.
- Vercel: For hosting. Jeg bruker Vercel for å møte kravet i oppgaven om at applikasjonen skal være deployet og tilgjengelig på nett.

Autentisering

Jeg har lagt mye vekt på autentisering og tilgangsstyring, da korrekt håndtering av roller er avgjørende for både sikkerhet og en god brukeropplevelse. Systemet skal skille mellom fire ulike roller: Gjest, Bruker, Arrangør og Admin.

Oversikt over roller og tilgang

Gjest (Anonym):

- Har lese-tilgang til alle offentlige arrangementer.
- Kan se detaljsider for hvert arrangement, men har ikke tilgang til personlige funksjoner eller administrasjonspaneler.

Bruker (Autentisert):

- Inkluderer alle gjesterettigheter.
- Har lese- og skrivetilgang til sin egen personlige favorittliste (`favorites`).

Arrangør:

- Inkluderer alle vanlige brukerrettigheter.
- Har tilgang til egne `/admin`-ruter for håndtering av innhold.
- Har full kontroll (CRUD) over arrangementer der de selv er satt som eier via `organizer_id`.

Admin:

- Systemets høyeste tilgangsnivå.

- Kan overstyre RLS-regler for å moderere eller fjerne upassende innhold på tvers av alle arrangører.
- Har generell brukerhåndtering.

Implementasjon av sikkerhet

For å sikre at applikasjonen er robust, er ikke sikkerheten begrenset til å kun skjule knapper i brukergrensesnittet. Tilgangen er implementert i to uavhengige lag:

1. Frontend: Next.js Middleware Før en side i det hele tatt rendres i nettleseren, sjekker Next.js Middleware brukerens sesjon og rolle. Forsøker en uautorisert bruker å aksessere en beskyttet rute (som `/admin`), blir de automatisk omdirigert til logg-inn-siden.

2. Backend: Supabase Row Level Security (RLS) validerer hver eneste database-forespørsel mot brukerens unike ID (UUID) og rolle, slik at man aldri kan manipulere data via API-kall uten tillatelse.

Arkitektur

For å sikre en rask og moderne webapplikasjon har jeg valgt en arkitektur som utnytter fordelene med Next.js App Router og Supabase. Målet er å flytte så mye som mulig av logikken og datahentingen til serveren for å gi brukeren en best mulig opplevelse.

1. Frontend

- **Rammeverk**: Next.js (TypeScript) – Benyttes for optimal ytelse gjennom Server Components og et robust rutesystem.
- **Styling**: Tailwind CSS – Brukes for å bygge et responsivt design raskt med utility-klasser.
- **UI-bibliotek**: Shadcn/UI – Gir tilgjengelige og ferdigstilte komponenter som sikrer universell utforming (UU).
- **Ikoner**: Lucide React – Et bibliotek med lette og konsistente ikoner for bedre brukervennlighet.
- **Linting/Formatering**: BiomeJS – Sørger for konsistent kodekvalitet og rask formatering av prosjektet.
- **Skjemahåndtering**: React Hook Form & Zod – Brukes i kombinasjon for å validere brukerinndata og sikre typesikkerhet før lagring i databasen.
- **Datahenting**: TanStack Query – Vurderes for håndtering av klient-tilstand og "Optimistic Updates" ved favoritt-markering.
- **Dato-håndtering**: date-fns – Brukes for å formatere og vise datoer på en enkel og forståelig måte for brukeren.

2. Backend

Som backend-løsning benyttes **Supabase**. Dette er en Backend-as-a-Service (BaaS) som samler database, autentisering og fillagring i en plattform. Dette valget er tatt for å sikre en rask utviklingsprosess og høy sikkerhet uten å måtte sette opp og drifte en egen server.

- **Database (PostgreSQL):** En relasjonsdatabase som brukes til lagring av arrangementer, brukerprofiler og favoritter.
- **Autentisering (Supabase Auth):** Håndterer sikker innlogging og registrering av brukere og arrangører.
- **Fillagring (Supabase Storage):** Brukes til lagring og servering av bilder som lastes opp til arrangementene.
- **Sikkerhet (RLS):** Row Level Security (RLS) brukes for å styre tilgangskontroll direkte i databasen, slik at brukere kun kan endre sine egne data.

Database struktur

For å sikre god dataintegritet og effektiv henting av informasjon, vil databasen bli bygget opp som en relasjonell database i PostgreSQL (via Supabase). Strukturen er designet for å håndtere relasjoner mellom brukere, deres roller og arrangementene som publiseres.

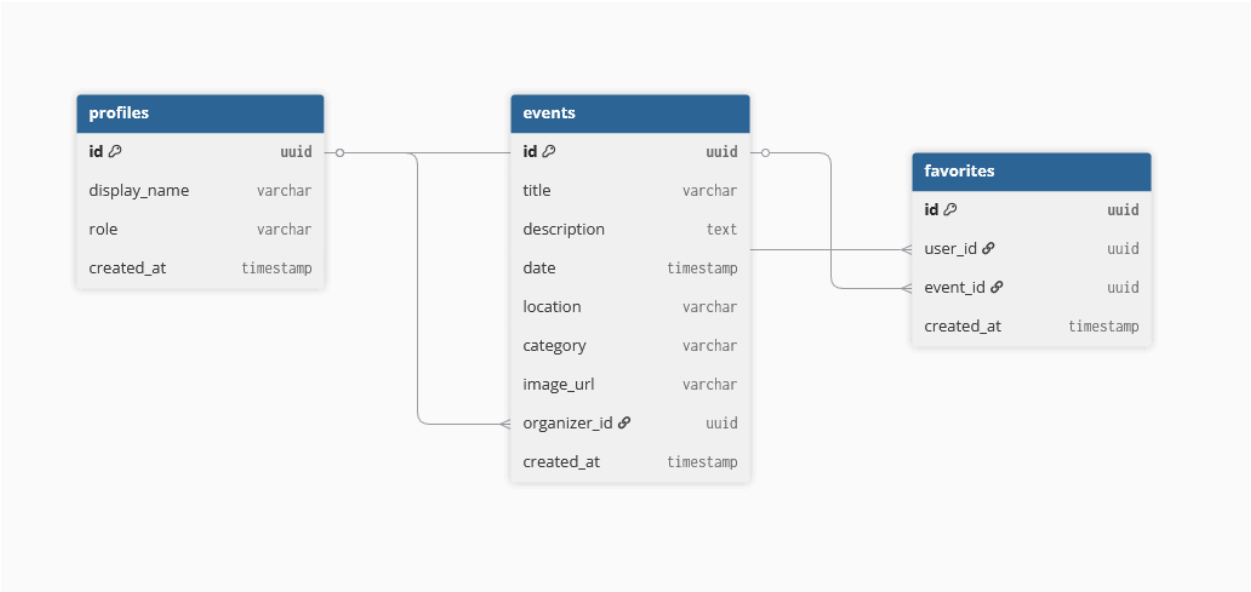
Tabelloversikt

Systemet består av følgende hovedtabeller:

- `profiles`: Inneholder utvidet informasjon om brukerne. Denne er koblet mot Supabase Auth sin interne brukertabell via en `UUID`.
- `events`: Kjernetabellen i applikasjonen som inneholder all data om arrangementene (tittel, beskrivelse, tid, sted, kategori og bilde-URL).
- `favorites`: En koblingstabell som gjør det mulig for brukere å lagre sine favorittarrangementer.

ER-Diagram (Entity Relationship)|

Diagrammet under viser hvordan tabellene er koblet sammen.



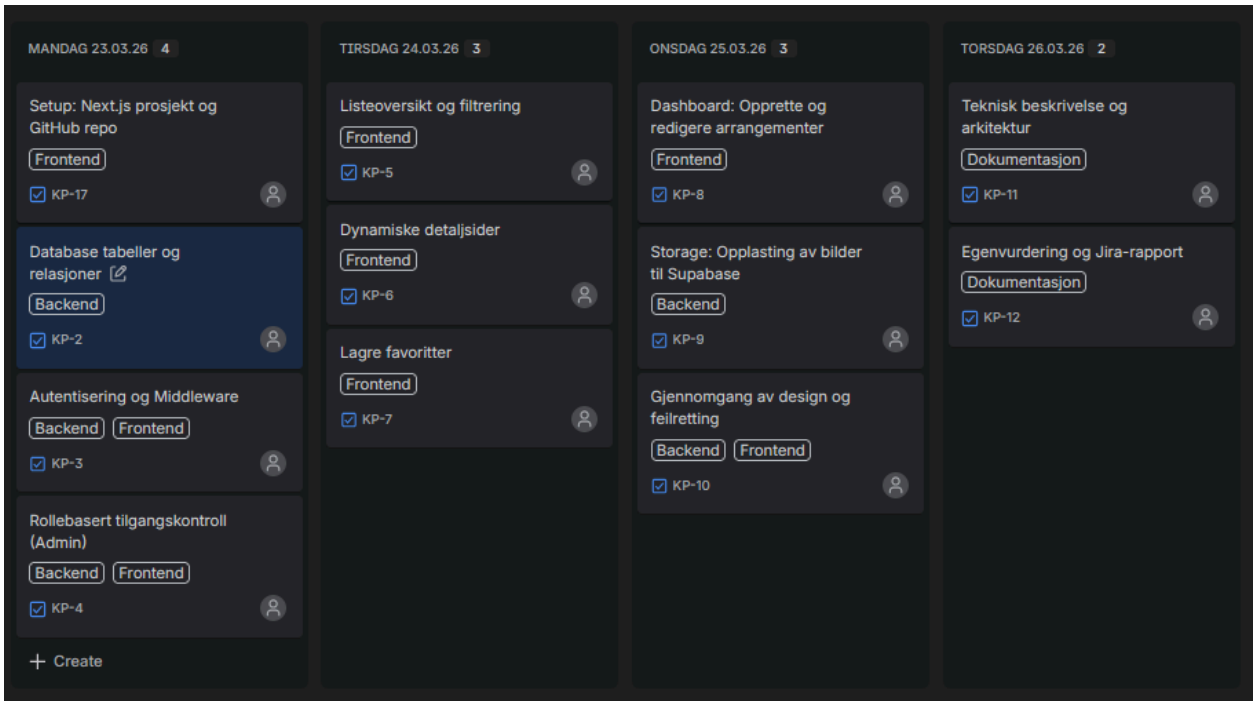
[Lenke til Diagram]

Relasjoner

For å få applikasjonen til å fungere som ønsket, er tabellene koblet sammen slik:

- **Arrangør til Arrangement**: Hvert arrangement er koblet til en profil. Dette gjør at vi alltid vet hvem som har lagt ut hva, og hvem som har rett til å endre innholdet.
- **Bruker til Favoritter**: Siden mange brukere kan like mange forskjellige arrangementer, bruker vi en koblingstabell. Dette gjør at brukeren enkelt kan hente ut sin egen liste over lagrede favoritter.

Dag til dag plan



Torsdag 19. mars

Dagen brukes til å sette meg grundig inn i oppgaveteksten og definere min tolkning av oppdraget. Kartlegge funksjonaliteten som kreves for "Den lokale arrangementsoversikten" og legge en strategi for hvordan de tekniske kravene skal løses.

Fredag 20. mars:

Dagen brukes til å ferdigstille planleggingsfasen og utarbeide den detaljerte dagsplanen for hele prosjektløpet. Bryte ned arbeidet i konkrete oppgaver og putte de inn i Jira for hver av dagene slik at alt ligger til rette for en effektiv utviklingsfase på mandag

Mandag 23. mars

Opprette GitHub-repo og initialisere Next.js-prosjektet. Sette opp Supabase-prosjektet og koble sammen frontend og backend. Opprette tabellene for `profiles`, `events` og `favorites` i databasen. Sette opp autentisering med Supabase Auth og lage en Middleware som passer på at bare de som er logget inn får se lukkede sider. Lage admin-dashboardet for å styre roller og tilgang slik at admin kan kontrollere hvem som er vanlig bruker, arrangør og admin.

Tirsdag 24. mars

Lage hovedsiden som henter og viser alle arrangementene. Lage søk og filtrering på kategorier, og sette opp egne sider for hvert arrangement (detaljsider). Bygge favoritt-systemet slik at brukerne kan trykke på et hjerte og lagre de arrangementene de liker

Onsdag 25. mars

Lage dashboardet der arrangører kan legge inn og endre sine egne arrangementer, og sette opp opplasting av bilder til Supabase. Bruke slutten av dagen på å teste hele appen grundig, finne og fikse feil og gjøre designet helt ferdig.

Torsdag 26. mars

Teste all funksjonalitet på nytt for å se at alt i oppgaven er gjort. Skrive teknisk dokumentasjon som forklarer hvordan jeg har laget appen og hvilke valg jeg har tatt. Dokumentere hvordan jeg har brukt Jira og skrive en egenvurdering om hva som har gått bra og hva som kunne vært bedre.

Fredag 27. mars

Gjøre ferdig dokumentasjonen og egenvurderingen. Forbrede en demo av applikasjonen og en gjennomgang av hvordan jeg har jobbet i løpet av uka. Presentere prosjektet for sensorene, svare på spørsmål om løsningene mine og levere inn koden og dokumentasjonen.

3. Gjennomføring

3.1 Torsdag 19. mars (dag 1)

Dagen startet med møtet med prøvenemnda for å gå gjennom oppgaven. Jeg satt igjen med et veldig positivt inntrykk. Oppgaven virket både spennende og relevant, og jeg så raskt at dette var en god mulighet til å vise frem bredden i det jeg kan innen både frontend, backend og sikkerhet.

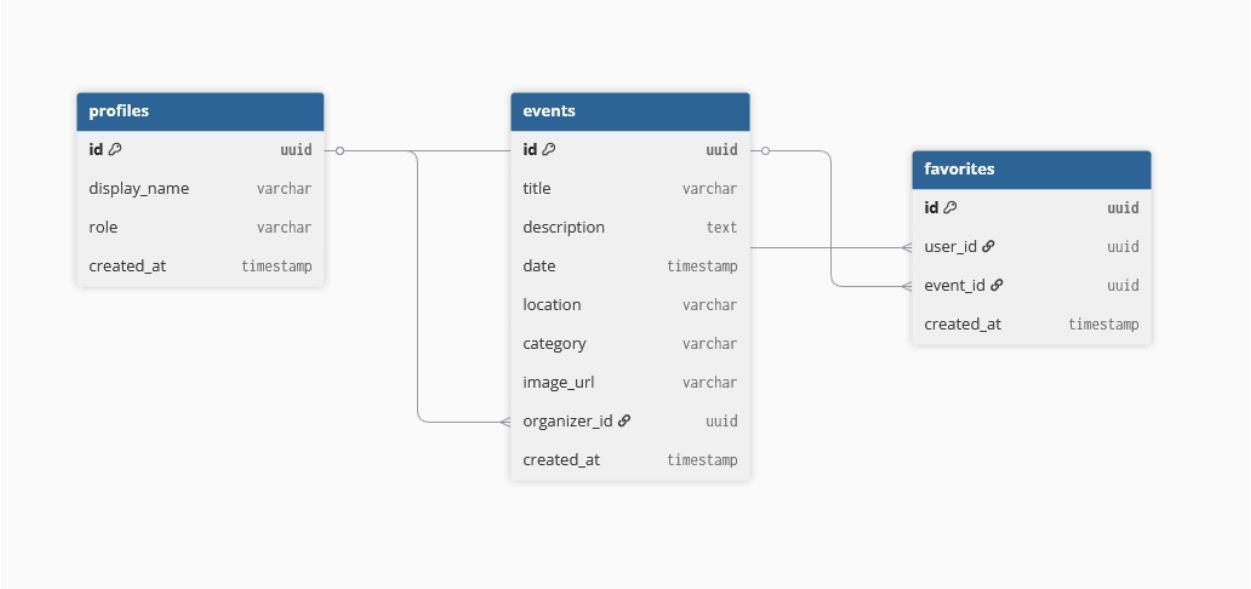
Etter møtet satte jeg i gang med planleggingen av prosjektet. Jeg startet med å skrive min tolkning av ptosjketet og nøyaktig hva løsningen måtte inneholde for å faktisk løse de behovene oppgaven krever. Dette gjorde jeg ikke bare for min egen del, men også for at nemnda senere skal kunne hvorfor jeg tokk de lagene jeg tok. Jeg begynte å legge rammene for techstacken, der valget falt på Next.js og Supabase. Jeg brukte også mye tid denne dagen på å tenke gjennom arkitekturen, spesielt hvordan jeg skulle håndtere autentisering og de ulike rollene (bruker, arrangør, admin) på en sikker måte helt fra start.

Egenvurdering og refleksjon

Alt i alt var dette en produktiv dag. Jeg brukte mye tid på å definere min egen tolkning av oppgaven, og jeg tror det gir meg et veldig godt grunnlag for resten av prosjektet. Jeg har også bestemt meg for techstack og begynt å skisse opp hvordan jeg skal løse autentiseringen med de forskjellige rollene, slik at sikkerheten er tenkt på helt fra starten av.

3.2 Fredag 20. mars (dag 2)

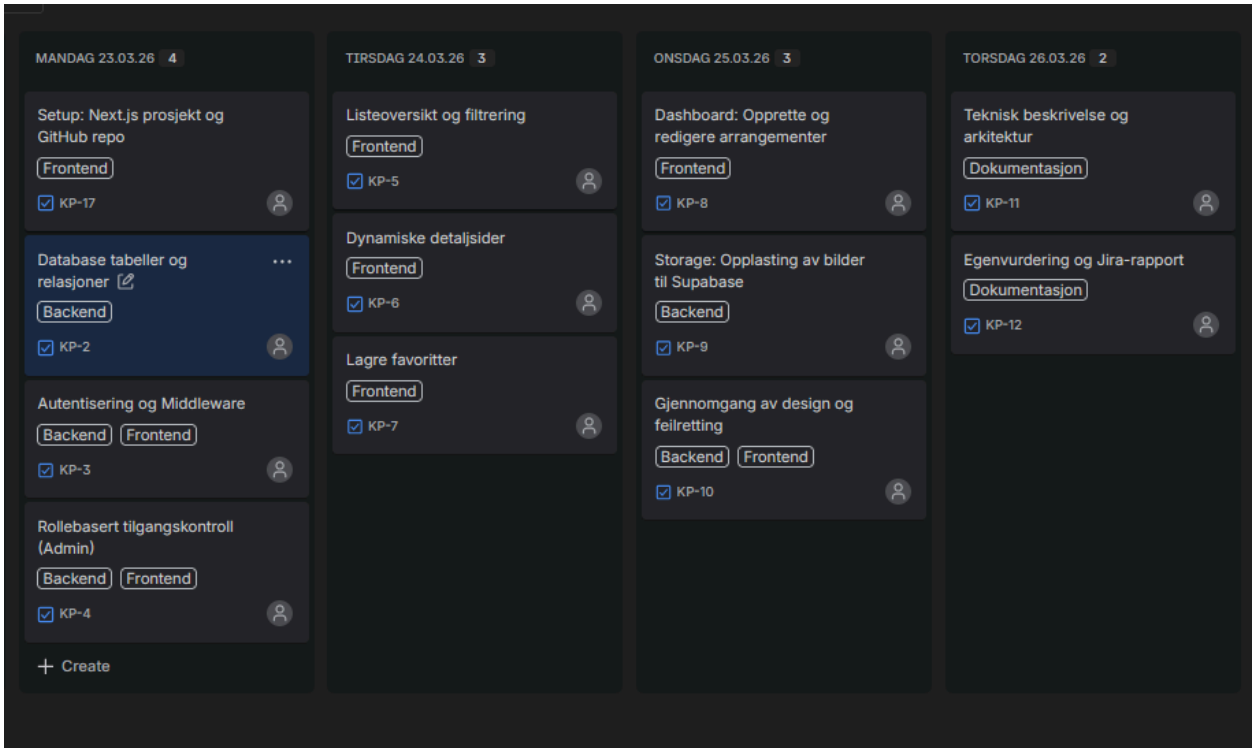
Fredagen ble brukt til å ferdigstille hele planleggingsdokumentet og sette opp rammene for selve gjennomføringen. Jeg tegnet opp et ER-diagram for å visualisere databasestrukturen, noe som hjalp meg med å se hvordan tabellene for profiler, arrangementer og favoritter skulle henge sammen.



ERD Diagramet

Det å få dette visuelt på plass tidlig gjorde det mye enklere å se for seg hvordan dataene skulle flyte gjennom applikasjonen.

Videre delte jeg opp hele prosjektet i mindre, håndterbare oppgaver som jeg la inn i Jira.



Jira boardet

Jeg brukte også slutten av dagen på å utarbeide en detaljert dag-til-dag-plan.

Her satte jeg opp faste mål for hver dag, slik at jeg visste nøyaktig når de ulike delene av logikken, som for eksempel RLS-sikkerhet og bildehåndtering, skulle prioriteres. Dette ga meg en trygghet på at jeg ville rekke å levere en komplett MVP innen fristen, samtidig som jeg hadde satt av tid til både feilsøking og dokumentasjon.

Egenvurdering og refleksjon

Jeg ble ferdig med planleggingen og føler meg trygg på valgene jeg har tatt for teknologi og rammeverk. Ved å få skisset opp databasetabellene og relasjonene tidlig, har jeg nå et veldig godt utgangspunkt for å starte selve utviklingen på mandag. Jeg har også fått laget en dag-til-dag-plan og definert konkrete oppgaver i Jira. Jeg tror oppdelingen er realistisk å gjennomføre på de tre dagene jeg har satt av, og jeg er veldig positiv til at selve gjennomføringen kommer til å gå bra.

3.3 Mandag 23. mars (dag 3)

Jeg startet dagen med å sette opp prosjektet i Supabase og opprette et repo på GitHub. Etter å ha klonet dette ned, satte jeg opp et Next.js-prosjekt med TypeScript, Tailwind CSS og BiomeJS. Jeg strukturerte mappene slik at det er et tydelig skille mellom de offentlige sidene og

de beskyttede områdene for admin og arrangør, slik at jeg har kontroll på tilgangsstyringen fra start.

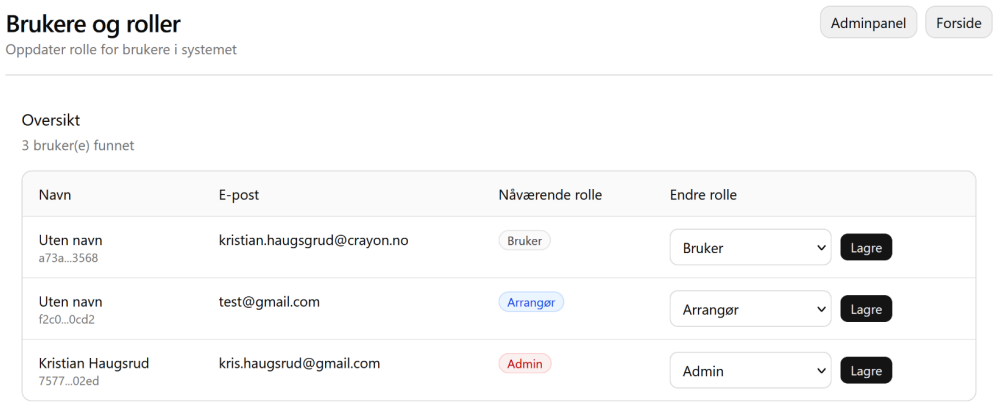
Etter oppsettet flyttet jeg fokus til databasen i Supabase. Jeg opprettet tabellene for profiler, arrangementer og favoritter. Her valgte jeg å lage en database-trigger som automatisk oppretter en profilrad hver gang en ny bruker registrerer seg. Dette gjorde jeg for å slippe å håndtere tomme profiler i koden senere; nå vet jeg at alle brukere alltid har en profil med en tildelt rolle.

Da datamodellen var klar, begynte jeg på autentiseringen. Her merket jeg raskt at Next.js er streng på hvordan man endrer cookies, og jeg fikk noen feilmeldinger da jeg prøvde å gjøre dette i en Server Component. Jeg løste dette ved å sette opp en middleware som tar seg av session-oppdateringer. Dette ga meg en stabil innloggingsflyt hvor brukeren blir værende logget inn mens de navigerer i appen.

For å sikre dataene i databasen, skrudde jeg på Row Level Security (RLS). Underveis fikk jeg en feilmelding om "infinite recursion" i en av mine policyer. Det skjedde fordi policyen prøvde å lese fra profil-tabellen samtidig som den sjekket tilgang til den. Jeg løste dette ved å sjekke brukerens rolle direkte fra JWT-tokenet i stedet for å gjøre et nytt oppslag i tabellen. Dette fjernet loopen og gjorde sikkerhetssjekken raskere.

Mot slutten av dagen satt jeg opp et enkelt adminpanel. Her kan jeg nå se en oversikt over alle registrerte brukere og endre rollene deres. Jeg har brukt ferdige komponenter fra shadcn/ui for å sikre at grensesnittet er ryddig og fungerer bra på både mobil og PC. Jeg har også valgt å bruke norske navn på alle ruter som `/favoritter` og `/arrangor` for at det skal være mest

mulig intuitivt for norske brukere.



Admin dashboardet hvor admin kan administrere brukere og roller

Egenvurdering og refleksjon

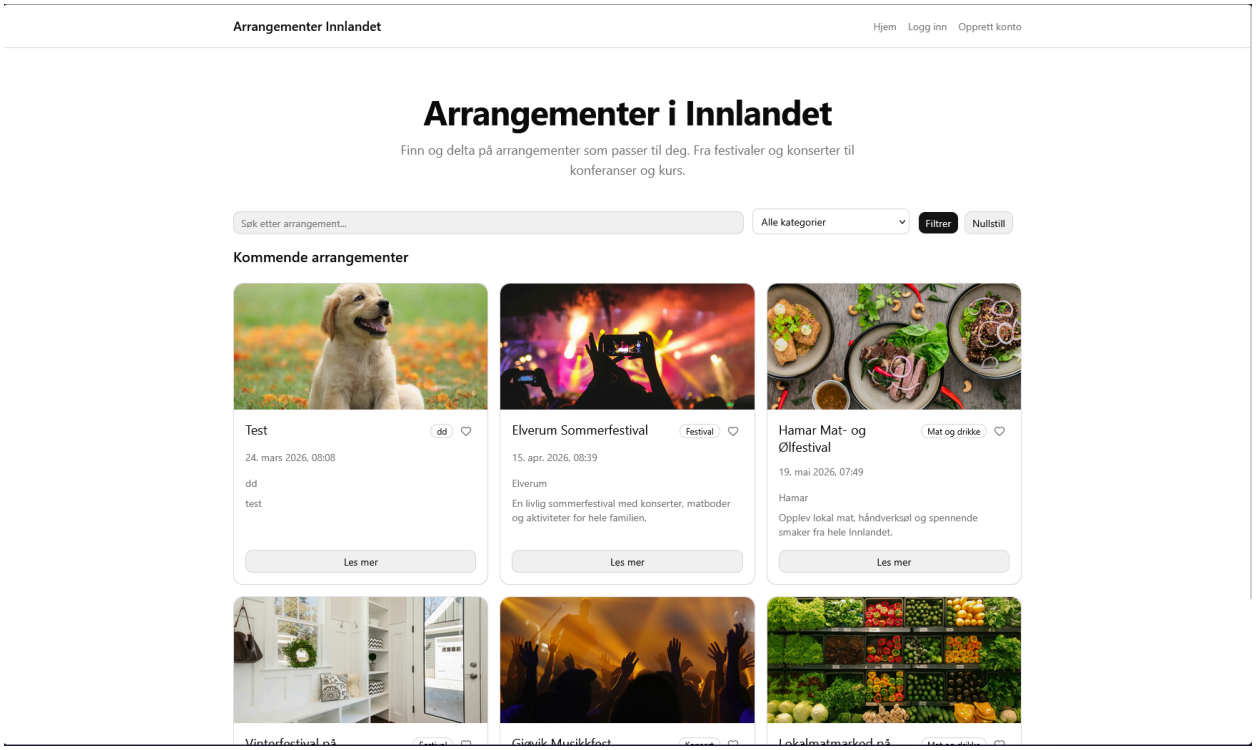
Alt i alt føler jeg at denne første dagen har vært veldig produktiv, og jeg har kommet i mål med hele oppsettet slik det var planlagt i Jira. Teknologien jeg har valgt fungerer godt sammen, og det at fundamentet med database og autentisering er på plass, gjør at jeg kan fokusere fullt på funksjonalitet i morgen.

Underveis i arbeidet med brukerroller oppdaget jeg imidlertid noe jeg ikke hadde planlagt godt nok i den opprinnelige skissen: en egen innstillingsside for brukerne. Jeg ser at det er behov for et sted hvor brukere selv kan redigere profilen sin eller slette kontoen sin for å ivareta personvern (GDPR). Etter å ha sett litt på hva som kreves for å implementere en trygg slettefunksjon i Supabase, ser jeg at dette vil ta en del tid som jeg ikke har satt av. Jeg har derfor bestemt meg for at dette mest sannsynlig faller utenfor scopet for denne oppgaven, da jeg må prioritere kjernefunksjonaliteten som arrangementer og favoritter for å bli ferdig i tide. Jeg velger heller å dokumentere dette som en planlagt videreutvikling.

3.4 Tirsdag 24. mars (dag 4)

I dag har jeg jobbet med listeoversikten, detaljsidene og favorittsystemet. Jeg startet med å bygge hovedsiden slik at den henter arrangementer fra databasen og viser dem i en ryddig kortvisning.

Etter at oversikten var på plass, implementerte jeg søk og filtrering. Jeg valgte å bruke query-parametre i URL-en for å styre dette. Det er en enkel og robust løsning som gjør at søkeresultatene oppfører seg forutsigbart, og det lar brukerne dele lenker til spesifikke søk. Etter det satt jeg opp de dynamiske detaljsidene for hvert enkelt arrangement.



Root siden

Den oppgaven som tok lengst tid i dag var favorittsystemet. Jeg har nå fått på plass en løsning der innloggede brukere kan lagre arrangementer de er interessert i. For å gjøre brukeropplevelsen bedre, la jeg inn en "optimistisk oppdatering". Det betyr at hjerteikonet reagerer med en gang brukeren trykker, i stedet for å vente på svar fra databasen. Hvis man ikke er logget inn, får man i stedet en tydelig beskjed om at man må logge inn for å lagre favoritter.

Jeg brukte også tid på å kontrollere at tilgangene i databasen fungerer riktig, slik at favoritter blir lagret for riktig bruker og ikke kan endres av andre.

Mot slutten av dagen deployet jeg til Vercel. Det gikk uten feil, og løsningen fungerer bra i produksjon også.

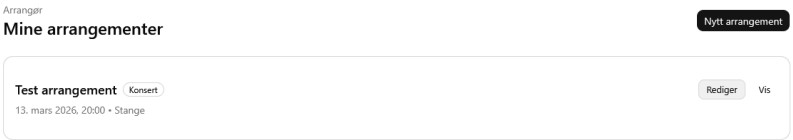
Egenvurdering og refleksjon

Dagen i dag har vært veldig produktiv, og jeg har fått ferdigstilt de viktigste funksjonene for en vanlig bruker.

Jeg hadde også en fin status med Nicklas, Martin og Markus. Vi snakket mest om planleggingen min, valgene jeg har tatt i databaseoppsettet, og sikkerhet rundt roller. Og jeg fikk noen gode innspill på hvordan man burde sikre roller slik at brukere ikke kan endre sine egne roller. Alt i alt er jeg fornøyd med dagen. Jeg har levert det som var planlagt, fått ting live, og fått nyttige innspill i statusmøtet som gjør prosjektet tydeligere videre.

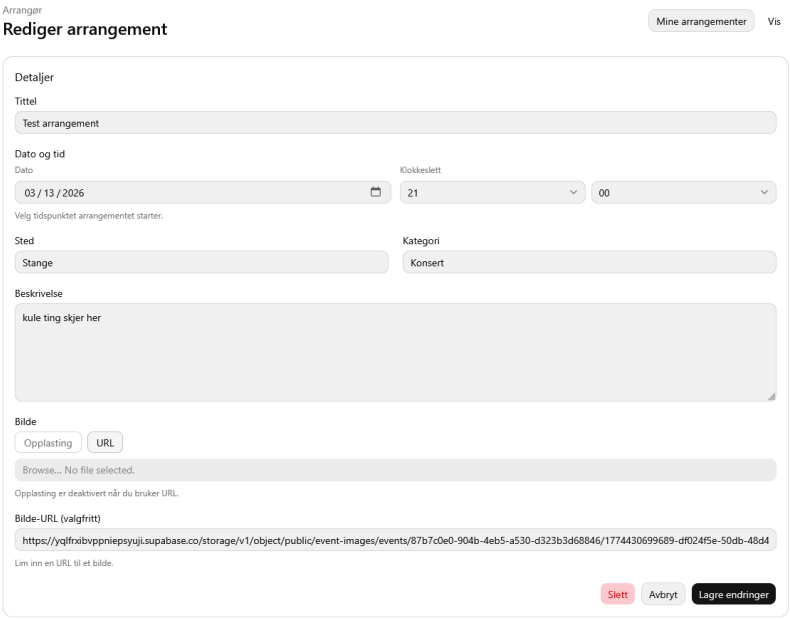
3.5 Onsdag 25. mars (dag 5)

I dag har jeg jobbet med å bygge ut arrangørdelen av løsningen. Hovedmålet var at de som er logget inn som arrangører, skal kunne administrere sine egne arrangementer direkte i applikasjonen.



Mine arrangementer side

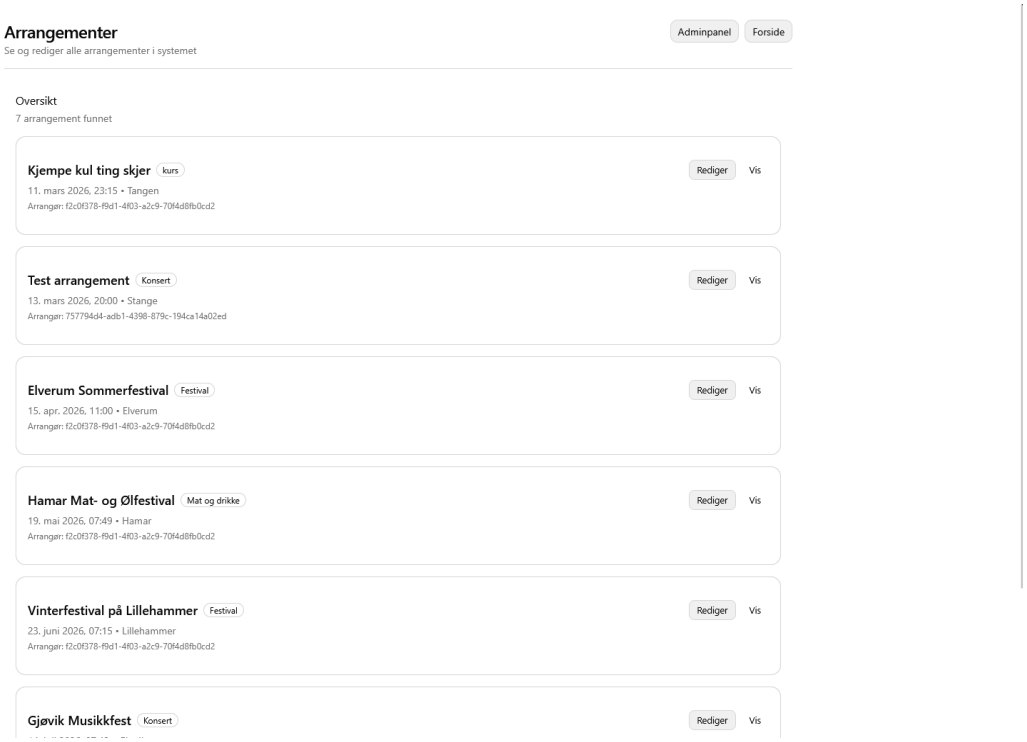
En stor del av dagen gikk med til å bygge skjemaer for å opprette og redigere arrangementer. For å sikre at dataene som sendes til databasen er korrekte, brukte jeg Zod til validering. Dette gjør at brukeren får tydelige feilmeldinger med en gang hvis de for eksempel glemmer en tittel eller velger en ugyldig dato.



Arrangement form der arrangører kan redigere arrangementene sine

Når det gjelder bilder, la jeg inn støtte for både direkte opplasting av bilder til Supabase Storage og bruk av eksterne bilde-URL-er. Jeg valgte å gjøre det slik at arrangøren må velge en av delene for å holde grensesnittet så enkelt og oversiktlig som mulig. For sletting av arrangementer valgte jeg å legge inn en bekreftelsesdialog der man må bekrefte handlingen før noe blir fjernet. Dette gjorde jeg for å unngå at brukeren sletter viktig innhold ved et uhell.

Utover dette har jeg utvidet admin-panelet slik at administratorer nå har full oversikt over alle arrangementer i systemet og kan moderere disse etter behov.



Admin oversikt over arrangementer

Resten av dagen brukte jeg på å gå over designet for å sikre at alt fra marger til knappestørrelser er konsistent på tvers av hele applikasjonen, enten man bruker mobil eller PC og skrev readme-en til prosjektet.

Egenvurdering og refleksjon

Dagen i dag har vært veldig produktiv, og det føltes godt å se hvordan rollestyringen jeg satte opp på mandag nå fungerer i praksis for arrangørene. Det som tok mest tid i dag var ikke selve databasen, men å få flyten i skjemaene og bildebehandlingen til å føles naturlig for brukeren uten at koden ble for komplisert.

Jeg føler nå at jeg har kommet i mål med alle punktene jeg hadde planlagt for gjennomføringen. Løsningen dekker alle de viktigste behovene i oppgaven, og jeg har en fungerende flyt for både vanlige brukere, arrangører og administratorer. Samtidig som jeg har kommet i mål, har jeg merket meg flere ting og funksjoner som kunne vært jobbet videre med for å gjøre løsningen enda bedre. Dette er punkter jeg kommer til å ta med og utdype i den endelige refleksjonen og i oversikten over videreutvikling senere i dokumentasjonen. Så jeg føler meg trygg for den siste finpussen i morgen.

3.5 Torsdag 26. mars (dag 6)

Dagen i dag har jeg i sin helhet brukt på å skrive ferdig dokumentasjonen, egenvurderingen og refleksjonene rundt prosjektet.

Jeg har også sett kritisk på mitt eget arbeid og skrevet ned hva jeg er fornøyd med, og hva jeg ser at jeg kunne gjort annerledes hvis jeg hadde hatt bedre tid. Dette innebærer blant annet de tekniske tingene jeg ikke rakk å bli helt ferdig med, som for eksempel en egen side for profilinnstillinger. Til slutt har jeg sørget for at all dokumentasjonen og veiledningene er ryddige og forståelige, slik at andre utviklere enkelt kan sette seg inn i hvordan applikasjonen er bygget opp og hvordan de kan jobbe videre med den.

3.6 Fredag 27. mars (dag 7)

I dag er siste dag av fagprøven, og fokuset har vært på de siste forberedelsene før presentasjonen.

Før jeg startet, fikk jeg beskjed om at sensorene ønsket å se på prosjektet og teste koden på egen hånd før selve fremvisningen. For å gi dem tilgang til databasen og autentiseringen måtte jeg dele miljøvariablene (`.env`-filen). Siden dette er sensitive data, valgte jeg å bruke tjenesten 1ty.me for å sende nøklene. Dette er en "self-destructing" lenke som gjør at informasjonen kun kan leses én gang. På denne måten sikret jeg at API-nøklene ikke blir liggende i chat-logger eller e-posthistorikk, noe som er helt i tråd med kravene til sikker håndtering av påloggingsopplysninger.

Resten av formiddagen har gått med til å gjøre meg klar til selve fremvisningen. Jeg har gått gjennom dokumentasjonen en siste gang for å sikre at alt henger sammen, og brukt tid på å strukturere hvordan jeg skal presentere løsningen. Jeg planlegger å bruke selve dokumentasjonen som et verktøy under presentasjonen for å vise frem både koden, databasestrukturen i Supabase og de tekniske valgene jeg har tatt dag for dag.

Egenvurdering og refleksjon Det føles bra å være i mål. Ved å dele miljøvariablene på en sikker måte fikk jeg vist at jeg tar sikkerhet på alvor også i praksis. Nå som dokumentasjonen er ferdigstilt, føler jeg meg trygg på at jeg har fått frem de viktigste tekniske løsningene, spesielt rundt RLS og rollestyring, som jeg gleder meg til å vise frem.

4. Egenvurdering

Jeg er stolt over alt jeg har oppnådd under denne prosessen og tenker selv at oppgaven gikk veldig bra. Det har vært en stor læringsprosess der jeg har fått testet meg på teknologi og verktøy jeg ikke var så kjent med fra før, noe som har gitt meg en dypere forståelse for faget. Det jeg er aller mest fornøyd med i applikasjonen er nok oppsettet med rollebasert tilgang. Jeg har satt opp enkel innlogging i Supabase tidligere, men det å dele brukere opp i roller som har forskjellige tilganger har vært veldig lærerikt. Jeg synes også jeg klarte å lage et enkelt, men veldig fint brukergrensesnitt og arrangørgrensesnitt som er intuitivt for brukerne å navigere i.

Det jeg er mest misfornøyd med er nok deler av arrangørgrensesnittet. Jeg er fornøyd med funksjonaliteten, men ser i ettertid at skjemaet for å opprette arrangementer kan være litt uoversiktlig. Det å dele det opp i flere steg hadde nok vært mer intuitivt for brukeren. Jeg hadde også likt å få inn søk og filtrering på oversikten for arrangører, siden det å bla gjennom mange arrangementer etter hvert kan bli rotete, men for en MVP synes jeg hele applikasjonen fungerer fint.

Hva jeg har lært

Jeg føler hele prosjektet har vært en god læringsprosess for meg. Jeg har gått dypere inn i verktøy og teknologi jeg har testet tidligere, men som jeg nå føler jeg har en mye bedre forståelse for.

Det området jeg har fått fordypet meg aller mest i er backend. Jeg bruker Supabase for backend og database. Jeg har brukt Supabase til enkle løsninger før og satt opp autentisering, men jeg har aldri gått så dypt i hva man må gjøre for å holde en tjeneste sikker. Jeg har lært mye om konsepter som RLS, CORS, triggere og policys.

Jeg har også fordypet meg i hvordan autentisering fungerer i Next.js. Spesielt har jeg blitt bedre på hvordan man bruker middleware for å beskytte ruter og kontrollere sesjoner på serversiden. Det å håndtere cookies og sørge for at brukeren blir omdirigert til riktig sted basert på rollen sin før siden i det hele tatt lastes inn.

Hva jeg kunne gjort annerledes

Jeg er egentlig veldig fornøyd med oppsettet og resultatet av applikasjonen. Ting som en innstillingsside og et mer intuitivt arrangørgrensesnitt hadde vært lurt å tenke på allerede i planleggingen. Jeg skulle også ønske jeg hadde satt opp noen enkle loaders for arrangementene. Jeg merker at bilder noen ganger er litt trege med å laste inn, og en loader ville fått siden til å virke mer responsiv, men jeg føler at jeg løste selve oppgaven fint.

Planleggingen

Planleggingen av prosjektet gikk bra, men jeg ser at jeg har forbedringspotensial. Her er noen ting jeg noterer meg:

- Glemte innstillingsside for brukere. Dette burde vært med i planen. En side for å redigere profil eller slette brukeren sin er viktig for personvern og GDPR.
- Litt for optimistisk på tid. Selv om jeg kom i mål, ser jeg at ting som feilhåndtering og UU-detaljer på skjemaer tar lengre tid enn man tror.
- Kunne vært flinkere til å koble læringsmålene direkte i planen. Jeg visste hva jeg skulle gjøre, men burde kanskje ha skrevet svart på hvitt i planen hvorfor jeg valgte for eksempel RLS for å dekke sikkerhetsmålet.

Jeg føler planleggingen ellers gikk ganske bra, og jeg er fornøyd med at jeg klarte å tenke meg fram til en god ERD for databaseoppsettet og relasjonene. Jeg burde nok sett litt dypere på ting som triggere og policys i Supabase i forkant, siden det til tider var vanskelig å forstå hvordan jeg skulle sette opp disse best mulig underveis i kodingen.

Gjennomføringen

Selve gjennomføringen føler jeg gikk veldig bra. Jeg klarte å holde fokus og følge dagsplanen jeg hadde satt opp for meg selv. Selv om det dukket opp noen tekniske utfordringer med databasen underveis, klarte jeg å løse disse systematisk uten at det gikk ut over fremdriften. Jeg føler jeg har hatt god kontroll på arbeidsprosessen fra start til slutt.

Plan for videre utvikling (Hvis jeg hadde hatt mer tid)

Brukerinnstillinger

Få på plass en egen side for brukerprofil der man kan endre navn, profilbilde og e-post. Det viktigste her er å implementere en funksjon for å slette egen konto. Dette er et krav i GDPR (retten til å bli glemt), og i en ferdig løsning må dette være på plass for at personvernet skal være ivaretatt.

Utvidet Admin-grensesnitt

Dagens admin-panel fungerer fint for få arrangementer, men når datamengden øker, trengs det bedre verktøy. Jeg vil legge til søk og filtrering for å kunne finne spesifikke arrangementer eller brukere på navn og e-post

Påmeldingssystem og sosial bevisføring

I stedet for å bare ha favoritter, vil jeg bygge et ordentlig påmeldingssystem. Dette innebærer en knapp for påmelding som lagres i en egen tabell, og en deltakerliste som viser en teller på arrangementet (for eksempel 12 personer er påmeldt). Dette skaper engasjement og gjør det mer fristende å delta.

Interaktivitet og kommentarer

Legge til et kommentarfelt på hvert arrangement. Dette lar brukere stille spørsmål til arrangøren og skaper et lite nettsamfunn rundt de lokale hendelsene. Her må det også legges inn logikk for at admin kan moderere innholdet.

Varslinger og påminnelser

Bruke Supabase Edge Functions til å sende ut automatiske varsler på e-post når et arrangement man er påmeldt til nærmer seg, eller gi beskjed til arrangøren når det kommer nye påmeldinger og kommentarer.

Kartvisning

Integrere Google Maps eller Mapbox for å vise arrangementene på et kart. Dette gjør det mye lettere for brukeren å se hva som skjer i nærheten av der de befinner seg.

Læremål

Planlegge, utvikle og dokumentere løsninger med innebygd personvern og sikkerhet

Tjenesten er satt opp med brukerinnlogging ved hjelp av Supabase Auth. Brukerdata i databasen er beskyttet med RLS, slik at bare brukeren selv eller admin kan se og redigere egne data. Dette sikrer at personvernet er ivaretatt helt nede på databasenivå, og at man ikke kan få tilgang til andres informasjon ved å manipulere forespørsler i nettleseren.

Utvikle og bruke dokumentasjon og veiledninger

Tjenesten har en README-fil som dokumenterer valg av teknologi og oppsett i både frontend og backend. Jeg har også dokumentert arkitekturvalg og hvordan man kloner og kjører prosjektet, slik at onboarding av nye utviklere vil gå fint. Under utviklingen har jeg tatt i bruk flere dokumentasjoner som blant annet:

- https://nextjs.org/docs
- https://supabase.com/docs
- https://zod.dev/

Dette er bare noen av dokumentasjonene jeg brukte under utviklingen.

Håndtere påloggingsopplysninger på en sikker og forsvarlig måte

Påloggingen håndteres gjennom Supabase. All påloggingslogikk, kryptering og registrering skjer gjennom deres hjelpfunksjoner. Dette er en trygg løsning som fjerner risikoen for at jeg ved et uhell misbehandler brukerdata. Passord blir kryptert og er aldri tilgjengelige for meg eller andre brukere. Jeg har også passet å ikke lagre mer data enn nødvendig, bortsett fra e-post og passord lagres kun brukerens rolle.

Velge og bruke relevante rammeverk og moduler til utvikling

Jeg har valgt Next.js som rammeverk og Supabase som backend fordi disse verktøyene tillater rask utvikling uten at det går på bekostning av sikkerhet eller ytelse. Ved å bruke moduler som

shadcn/ui har jeg kunnet bygge et profesjonelt grensesnitt effektivt, mens Zod har blitt brukt til skjemavalidering for å sikre at dataene som sendes til databasen alltid er i riktig format.

Feilsøke kode og rette feil i algoritmer og kode

Jeg føler att feilsøking av kode og å rette feil er noe som skjer naturlig når man utvikler en tjeneste. Det er ikke så mye forklaring jeg føler er nødvendig å skrive ned hvorfor dette punktet er oppfylt

Utvikle og tilpasse brukergrensesnitt som ivaretar krav til universell utforming

For å sikre at tjenesten er universelt utformet uten at det stjal for mye tid fra utviklingen, valgte jeg å bruke komponentbiblioteket shadcn/ui. Siden disse komponentene er bygget på Radix UI, får jeg kritisk funksjonalitet som tastaturnavigasjon og støtte for skjermlesere helt gratis. Dette gjør at jeg kan levere en profesjonell løsning som følger kravene på en effektiv måte, i stedet for å bruke tid på å kode tilgjengelighet fra bunnen av. Jeg har i tillegg passet på at fargekontraster og responsivt design fungerer på alle skjermstørrelser.